

Advanced Python Programming for Data Science

CH – 06

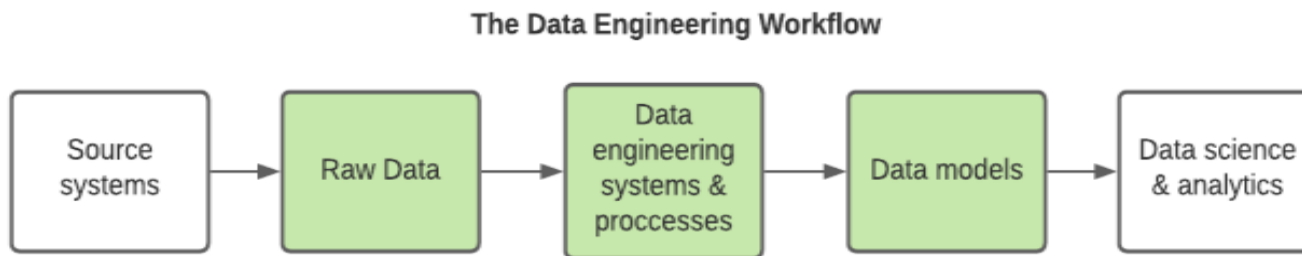
Data Engineering and Automation

Baikuntha Acharya (baikunth2a@gmail.com)

Senior Lecturer, Sagarmatha Engineering College, Sanepa, Lalitpur

Overview of data engineering

- ✓ Data engineering is the process of designing systems that collect, process, store, and deliver data for analysis and machine learning.
 - Raw data is often **messy, incomplete, and distributed across multiple sources**.
- ✓ It ensures clean, reliable, and well-structured data.
- ✓ Data Pipeline Concept:
 - A **data pipeline** is a sequence of steps that moves data from raw sources to analysis-ready datasets.
- ✓ Data Engineering Workflow:
 - Raw data is ingested from source systems → stored and processed → served as conformed data models → datasets to consumers (*such as data scientists and data analysts*).



Overview of data engineering (Contd..)

- ✓ The data engineering lifecycle is the series of stages that turn raw data ingredients into a useful end product, ready for consumption by analysts, machine learning engineers, etc.
- ✓ The **major stages of the data engineering** lifecycle:
 - Generation: source systems
 - Ingestion
 - Storage
 - Transformation
 - Serving data

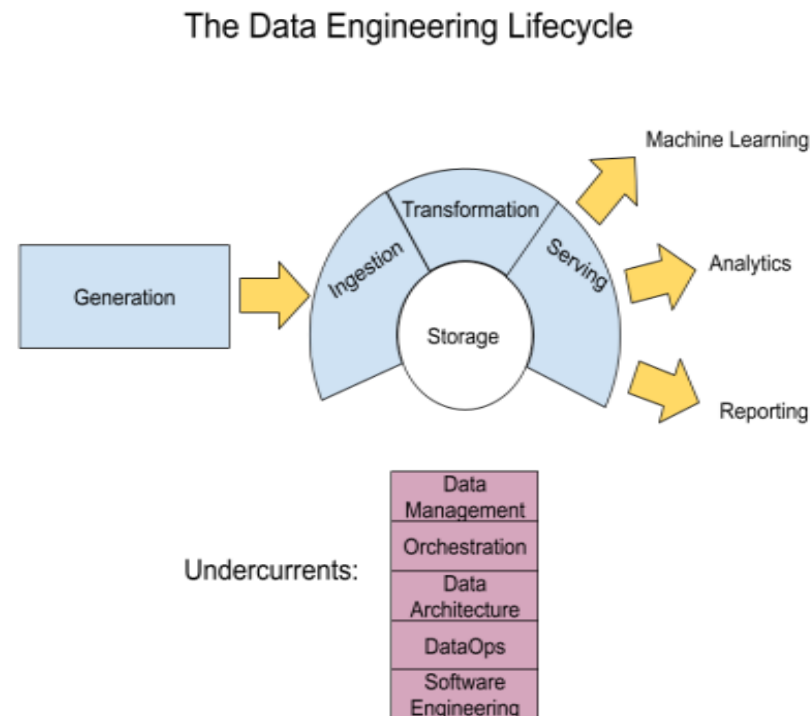


Figure: Components and undercurrents of the data engineering lifecycle

Overview of data engineering (Contd..)

- ✓ Simple example pipeline:

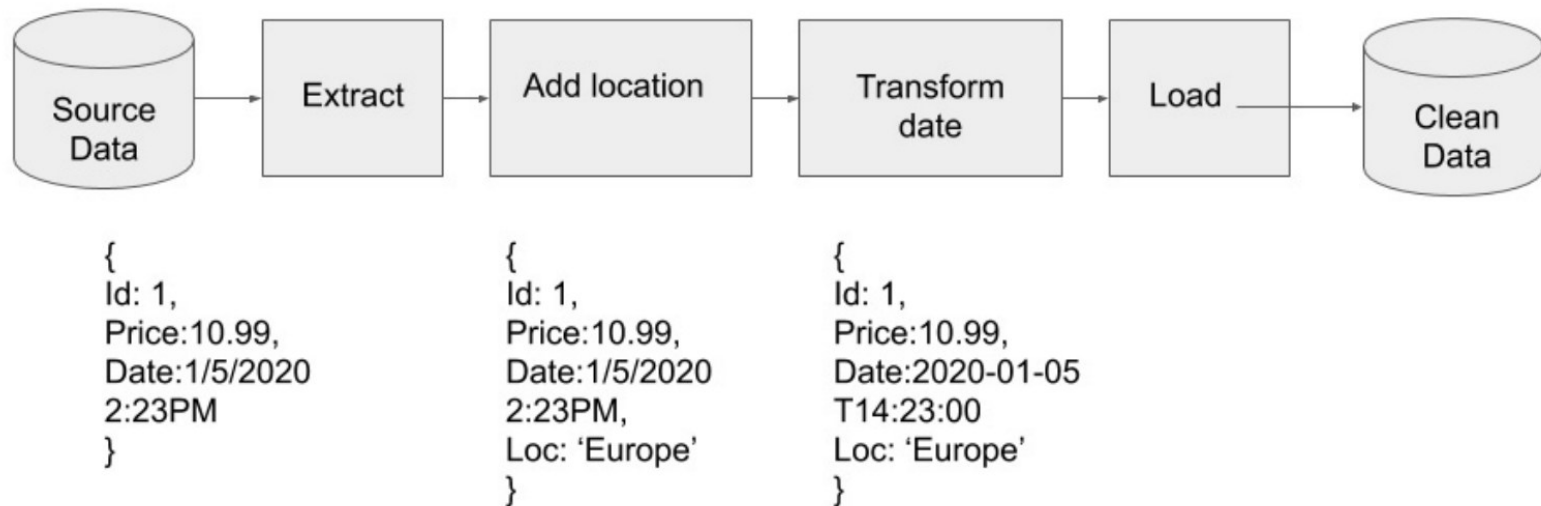


Figure 1.1 – A pipeline that adds a location and modifies the date

Role of Data Engineering in the Data Lifecycle

✓ Data Collection

- Collects data from multiple sources:
 - Websites & apps (user activity, transactions)
 - Databases (customer and sales records)
 - IoT devices & sensors (GPS, machine data)
 - Documents & images (PDFs, reports)
- Ensures **efficient extraction**, no data loss, and no duplication

✓ Data Storage

- Selects appropriate storage solutions:
 - **Data Warehouses** → structured, analytics-ready data
 - **Data Lakes** → raw and unstructured data
- Ensures data is **scalable, secure, and accessible**

Overview of data engineering (Contd..)

✓ Data Processing

- Cleans and transforms raw data:
 - Deduplication
 - Normalization (formats, units)
 - Aggregation (summaries)
- Converts messy data into **reliable, usable datasets**

✓ Data Integration

- Combines data from multiple systems into a **single source of truth**
- Eliminates data silos
- Enables cross-department, data-driven decisions

✓ **Data Quality & Governance**

- Maintains data accuracy and integrity through:
 - Validation checks
 - Security and encryption
 - Regulatory compliance (GDPR, HIPAA)
- Prevents incorrect analysis and compliance risks

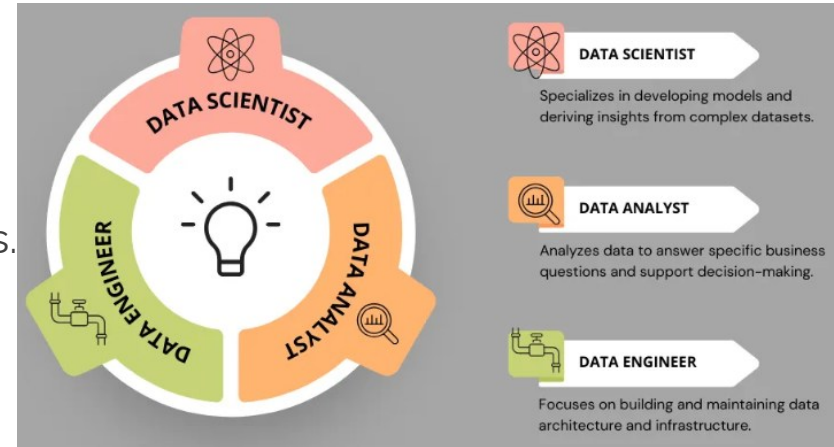
✓ **Data Provisioning**

- Makes clean data available via:
 - Dashboards & reports (Power BI, Tableau)
 - APIs for real-time access
 - Data models for ML & AI
- Turns data into actionable insights

Overview of data engineering (Contd..)

✓ Data Engineer: Core responsibilities:

- Building and maintaining data pipelines (ETL: Extract, Transform, Load).
- Managing databases and data warehouses.
- Ensuring data quality, consistency, and security.
- Optimizing data infrastructure for scalability and performance.



✓ Data Scientists: Core responsibilities:

- Developing machine learning models to predict trends and automate decision-making.
- Analyzing large, complex datasets to uncover insights.
- Designing experiments to test hypotheses and improve business strategies.
- Writing code (often in Python, R, or SQL) to manipulate and model data.

Overview of data engineering (Contd..)

✓ **Data Analyst: Core responsibilities:**

- Cleaning, organizing, and summarizing data (descriptive statistics).
- Conducting exploratory data analysis to identify trends and relationships.
- Creating data visualizations (charts, dashboards, reports) to communicate findings.
- Using tools like Excel, SQL, Tableau, or Power BI to make insights digestible.

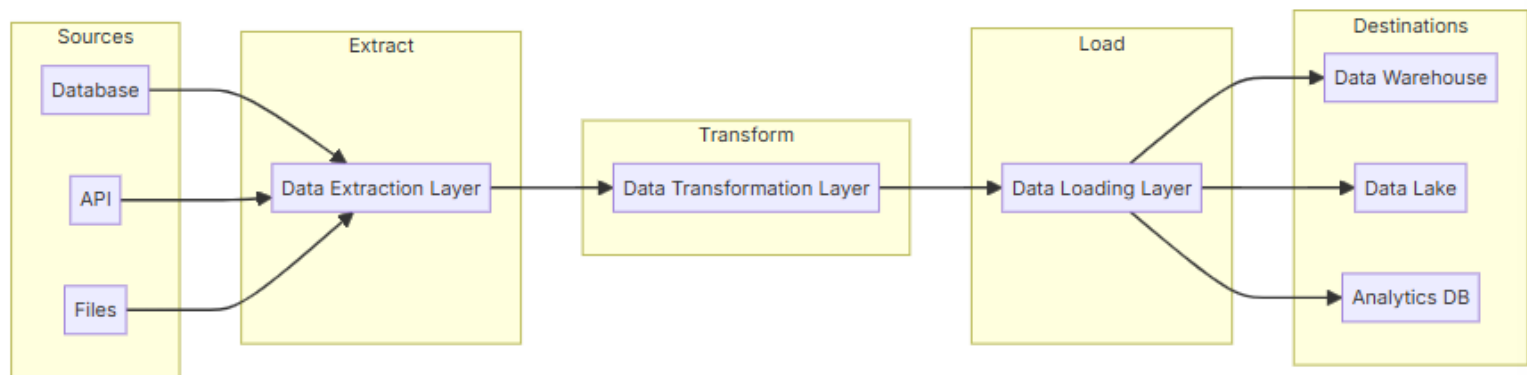
How do these roles work together?

- **Data Engineers** build the foundation, the pipelines and systems that collect, process, and store data.
- **Data Scientists** create the blueprints, applying algorithms and advanced analytics to extract deep insights.
- **Data Analysts** furnish the house, turning data into easy-to-understand reports, dashboards, and visuals for decision-makers.

Designing and implementing ETL pipelines

- ✓ Data engineering relies heavily on ETL (Extract, Transform, Load) pipelines to move and process data from source systems into data warehouses and analytics platforms.
 - It describes the process of moving data from source systems to destination systems while changing its format.
- ✓ A well-designed ETL pipeline ensures data quality, reliability, and scalability.

High-level architecture of an ETL pipeline



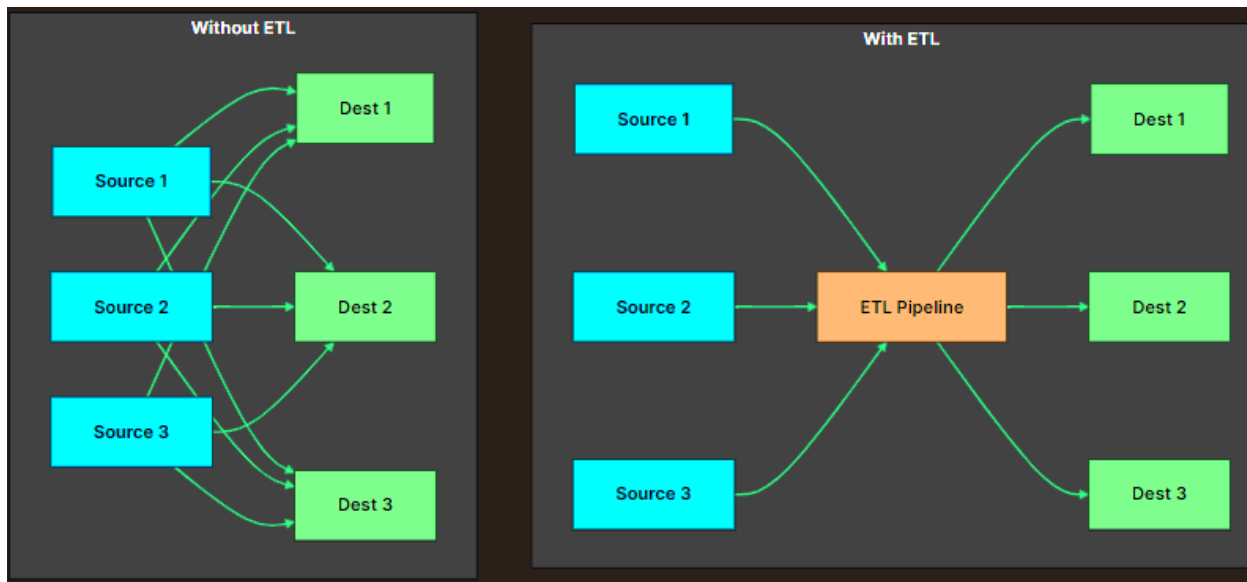
Designing and implementing ETL pipelines (Contd..)

Phase	Purpose	Example
Extract	Read data from source systems	Query production database, call APIs, read files
Transform	Convert data to desired format	Clean, validate, aggregate, join, denormalize
Load	Write data to destination system	Insert into warehouse, update tables

Designing and implementing ETL pipelines (Contd..)

✓ Why ETL Matters?

- Without ETL, every system would need to:
 - Understand every source system's format
 - Handle all data quality issues itself
 - Maintain connections to all data sources
 - Compete with production/ data source systems for resources
- ETL centralizes these concerns:



Designing and implementing ETL pipelines (Contd..)

Extraction Phase

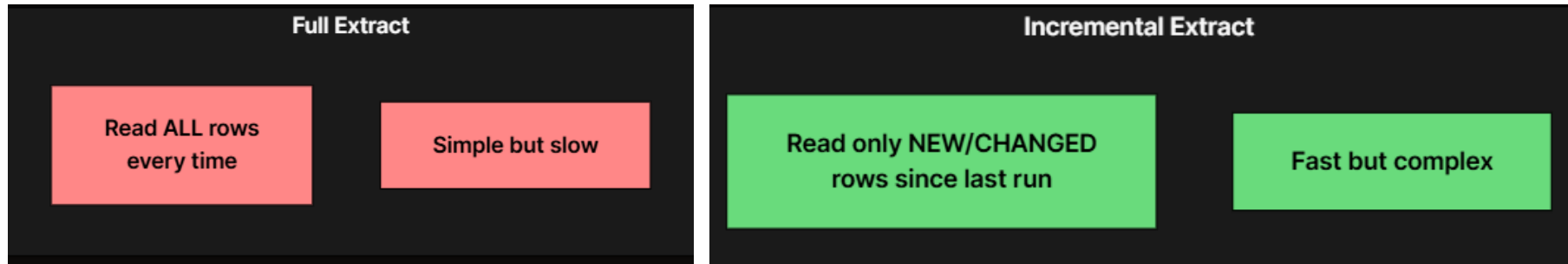
- ✓ The Extract phase reads data from source systems. This sounds simple but involves significant complexity.
- ✓ Common Data Sources

Source Type	Examples	Challenges
Relational databases	MySQL, PostgreSQL, Oracle	Connection limits, query impact on production
NoSQL databases	MongoDB, Cassandra, DynamoDB	No standard query language, denormalized data
APIs	REST, GraphQL, SOAP	Rate limits, pagination, authentication
Files	CSV, JSON, Parquet, logs	Schema variations, encoding issues
Message queues	Kafka, RabbitMQ	Ordering, exactly-once processing
SaaS platforms	Salesforce, Stripe, HubSpot	API limits, data access restrictions

Designing and implementing ETL pipelines (Contd..)

Extraction Phase (Contd..)

✓ Full Extract vs Incremental Extract



Method	Pros	Cons
Full extract	Simple, always consistent	Slow, wastes resources, strains source
Incremental extract	Fast, minimal source impact	Requires change tracking, can miss deletes

Extraction Phase (Contd..)

Incremental Extract Strategies

- ✓ **Timestamp-based:** Query rows where **updated_at > last_run_time**

```
SELECT * FROM orders
WHERE updated_at > '2024-01-15 00:00:00'
```

- ✓ **Sequence-based:** Track a monotonically increasing ID

```
SELECT * FROM events
WHERE id > 1000000
```

- ✓ **Change Data Capture (CDC):** Read from database transaction log

- CDC is the most reliable for incremental extraction because it captures all changes, including deletes, without modifying the source schema.

```
MySQL binlog, PostgreSQL WAL, Debezium
```

Extraction Phase (Contd..)

✓ Handling Source System Impact

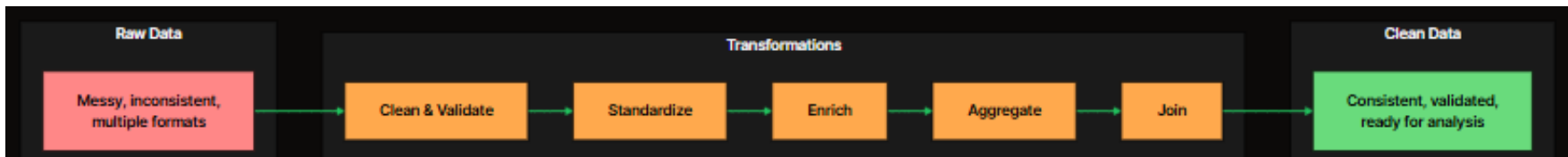
- Extracting data should not degrade production performance:

Strategy	How It Works
Read replicas	Query a replica instead of primary
Off-peak scheduling	Run extracts during low-traffic hours
Rate limiting	Limit query concurrency and batch size
Snapshots	Extract from database snapshots
CDC	Read from log without querying tables

Designing and implementing ETL pipelines (Contd..)

Transformation Phase

- ✓ The Transform phase converts raw data into the format needed by destination systems. This is where most of the business logic lives.
- ✓ **Common Transformations**



Designing and implementing ETL pipelines (Contd..)

Transformation Phase (Contd..)

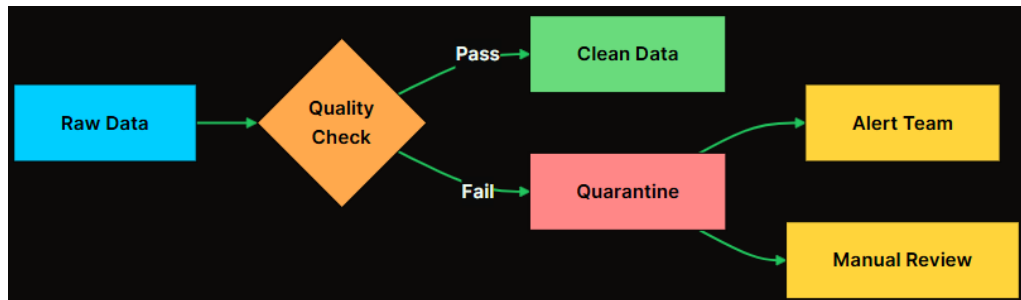
✓ Transformation Types

Type	Description	Example
Cleaning	Fix data quality issues	Remove nulls, fix encoding, trim whitespace
Validation	Ensure data meets requirements	Check required fields, validate formats
Standardization	Make data consistent	Convert dates to UTC, normalize phone numbers
Enrichment	Add derived data	Calculate age from birthdate, add geo data from IP
Aggregation	Summarize data	Sum daily sales, count monthly users
Joining	Combine data sources	Join orders with customers
Filtering	Remove unwanted data	Exclude test accounts, filter by date range
Deduplication	Remove duplicates	Dedupe by user email
Denormalization	Flatten for analytics	Embed customer data in order records

Transformation Phase (Contd..)

✓ Data Quality Checks

- Transform is where you enforce data quality:



✓ Common quality checks:

- **Null checks:** Critical fields must have values
- **Type checks:** Dates are dates, numbers are numbers
- **Range checks:** Values within expected bounds
- **Format checks:** Email, phone, postal code formats
- **Referential checks:** Foreign keys exist in parent tables
- **Uniqueness checks:** Primary keys are unique

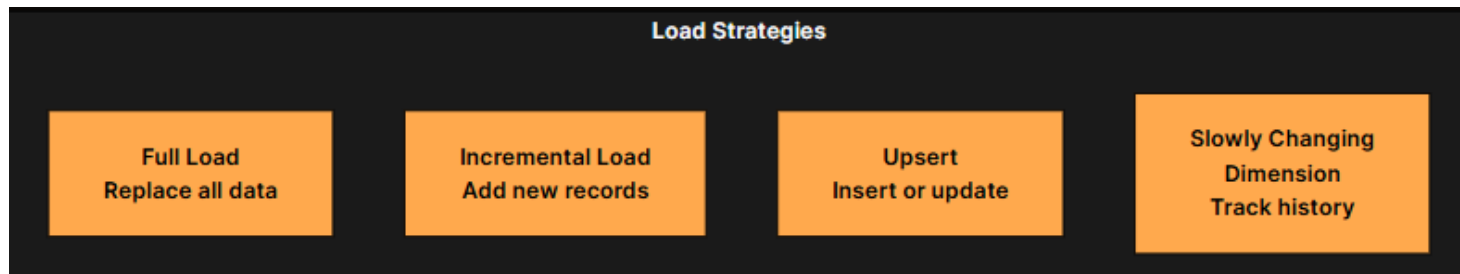
Designing and implementing ETL pipelines (Contd..)

Load Phase

- ✓ The Load phase writes transformed data to destination systems. The strategy depends on the destination type and use case.

- ✓ **Load Strategies**

A Dimension Table is a reference table in a dimensional model that holds **descriptions and static information about data**.



Strategy	When to Use	How It Works
Full load	Small tables, complete refresh needed	Truncate and reload
Incremental append	Event/transaction tables	Insert new rows only
Upsert (merge)	Dimension tables	Insert new, update existing

Designing and implementing ETL pipelines (Contd..)

Load Phase (Contd..)

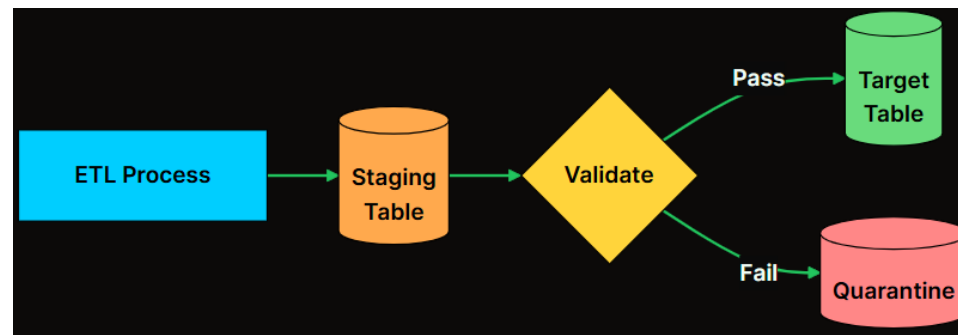
✓ Bulk Loading Techniques

- For large data volumes, row-by-row inserts are too slow:

Technique	Description	Speed
Batch INSERT	Multi-row INSERT statements	10x faster than single
COPY command	Load from files (Postgres, Redshift)	100x faster
Bulk loader	Native database utilities	Fastest
Parallel loading	Multiple concurrent loaders	Scales with connections

✓ Staging Tables

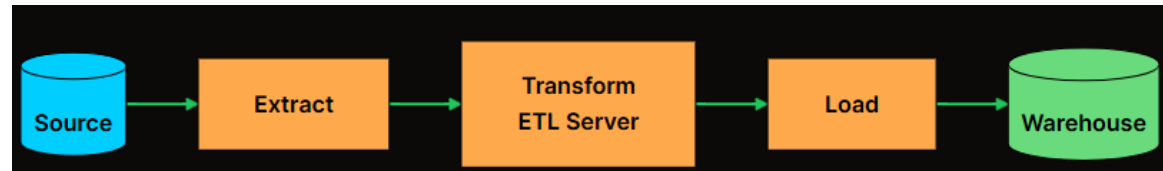
- Temporarily store raw or lightly processed data extracted from source systems.



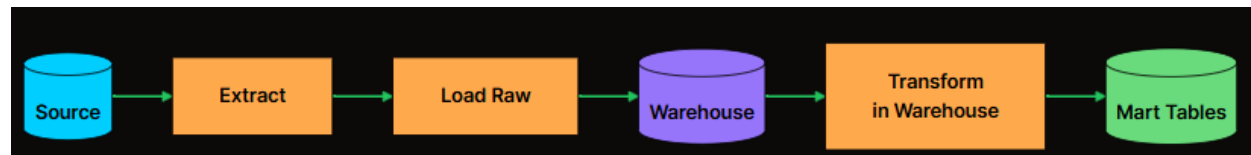
Designing and implementing ETL pipelines (Contd..)

✓ **ETL vs ELT** - Modern data platforms have shifted from ETL to ELT.

- **ETL: Transform Before Load** : Transformations happen on ETL servers before loading.



- **ELT: Transform After Load** : Raw data is loaded first, transformations happen in the warehouse.



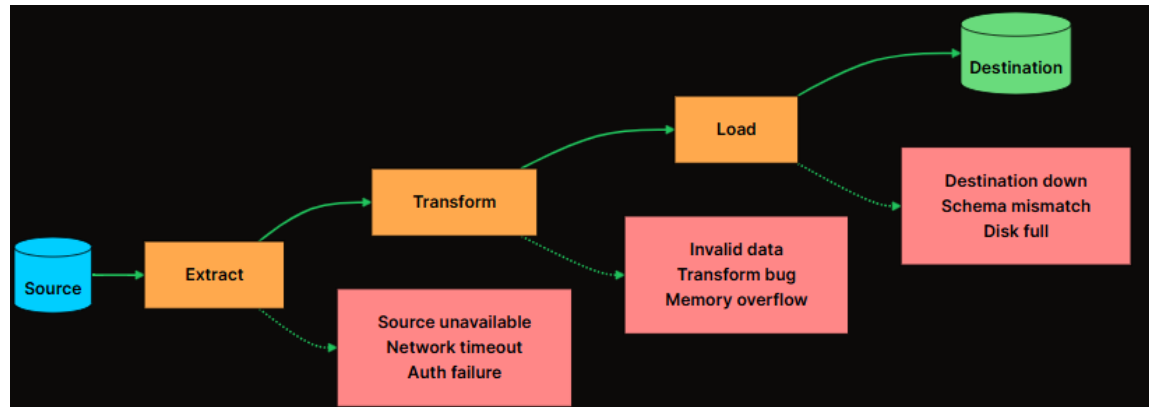
Aspect	ETL	ELT
Transform location	ETL server	Data warehouse
Compute cost	ETL infrastructure	Warehouse compute
Raw data	Not preserved	Preserved in warehouse
Flexibility	Changes require ETL updates	Transform with SQL anytime
Best for	Structured, well-defined transformations	Exploration, evolving requirements
Tools	Informatica, Talend, SSIS	dbt, Dataform, warehouse SQL

Designing and implementing ETL pipelines (Contd..)

✓ Handling Failures

- ETL pipelines fail. Networks drop, sources become unavailable, data has unexpected formats. Robust pipelines handle failures gracefully.

✓ Failure Points



✓ Failure Handling Strategies

Strategy	Description
Retry with backoff	Retry transient failures with increasing delays
Dead letter queue	Send failed records to separate storage for investigation
Checkpointing	Save progress so jobs can resume from last success
Idempotency	Design jobs to be safely re-run without side effects
Partial success	Allow job to complete with some failures, report issues
Circuit breaker	Stop trying after repeated failures to prevent cascade

✓ Idempotent Loads

- An idempotent operation produces the same result whether run once or many times:

```
1 NOT idempotent:
2   INSERT INTO orders (id, amount) VALUES (1, 100)
3   Running twice creates duplicate
4
5 Idempotent:
6   INSERT INTO orders (id, amount) VALUES (1, 100)
7   ON CONFLICT (id) DO UPDATE SET amount = 100
8   Running twice produces same result
```

✓ Making ETL idempotent allows safe retries:

- Use **upserts** instead of inserts
- Track processed records externally
- Use unique constraints to prevent duplicates

ETL Tools

✓ Traditional ETL Tools

Tool	Description	Best For
Informatica	Enterprise ETL, drag-and-drop	Large enterprises, complex mappings
Talend	Open source + enterprise	Cost-conscious organizations
SSIS	Microsoft SQL Server	Microsoft ecosystem
DataStage	IBM enterprise ETL	IBM shops

✓ Modern ELT Tools

Tool	Description	Best For
dbt	SQL-based transformations	Warehouse-centric, version control
Dataform	SQL transformations (Google)	BigQuery users
Fivetran	Managed data ingestion	Quick setup, SaaS sources
Airbyte	Open source data integration	Custom connectors, self-hosted
Stitch	Managed data pipelines	Simple SaaS to warehouse

ETL Pipeline Implementation Example

Python Libraries for ETL

✓ Data Extraction Libraries

- **requests** – fetch data from REST APIs
- **sqlalchemy** – database connectivity and ORM
- **pymongo** – connect to MongoDB (NoSQL)
- **boto3** – access AWS cloud storage (S3)
- **kafka-python** – read streaming data from Kafka

✓ Data Transformation Libraries

- **pandas** – data cleaning, aggregation, joins
- **numpy** – numerical transformations
- **pydantic** – schema validation
- **PySpark**: The Python API for Apache Spark, used for large-scale, distributed data processing and analysis.

✓ Data Loading Libraries

- **sqlalchemy** – write to relational databases
- **pandas.to_sql()** – quick loading to SQL systems
- **pyarrow / fastparquet** – efficient columnar storage
- **boto3** – upload processed data to cloud storage

✓ Workflow & Pipeline Support

- **schedule** – lightweight job scheduling
- **logging** – pipeline monitoring
- **airflow / prefect** – workflow orchestration (industry tools)

ETL Pipeline Implementation Example

Connecting to Different Data Sources

1. Connecting to Relational Databases

</> Python

```
import pandas as pd
from sqlalchemy import create_engine

# connect to PostgreSQL database
engine = create_engine("postgresql://user:password@localhost:5432/db")

# extract data
df = pd.read_sql("SELECT * FROM orders", engine)
```

2. Extracting Data from REST APIs

</> Python

```
import requests
import pandas as pd

url = "https://api.example.com/orders"
response = requests.get(url)

# convert JSON to dataframe
data = pd.DataFrame(response.json())
```

3. Reading Data from Files (CSV / JSON)

</> Python

```
import pandas as pd

orders = pd.read_csv("orders.csv")
customers = pd.read_json("customers.json")

print(orders.head())
```

4. Reading Data from NoSQL Databases

</> Python

```
from pymongo import MongoClient
import pandas as pd

client = MongoClient("mongodb://localhost:27017")
db = client["shop"]

data = list(db.orders.find())
df = pd.DataFrame(data)
```

ETL Pipeline Implementation Example

Example 1 — Basic Modular ETL Pipeline

```
import pandas as pd

def extract():
    return pd.read_csv("orders_raw.csv")      # read source data

def transform(df):
    df = df.dropna()                          # remove missing records
    df["revenue"] = df["price"] * df["qty"]    # create derived column
    return df

def load(df):
    df.to_csv("orders_clean.csv", index=False) # store processed data

def run_pipeline():
    data = extract()
    data = transform(data)
    load(data)

run_pipeline()
```

ETL Pipeline Implementation Example

Example 2 — Multi-Source ETL Pipeline

```
import pandas as pd, requests
from sqlalchemy import create_engine

def extract_orders():
    return pd.DataFrame(requests.get("https://api.example.com/orders").json())

def extract_customers():
    engine = create_engine("sqlite:///sales.db")
    return pd.read_sql("SELECT * FROM customers", engine)

def transform(orders, customers):
    df = orders.merge(customers, on="customer_id") # combine datasets
    df["revenue"] = df["price"] * df["qty"]
    return df

def load(df):
    engine = create_engine("sqlite:///warehouse.db")
    df.to_sql("sales_fact", engine, if_exists="replace", index=False)

orders = extract_orders()
customers = extract_customers()
clean = transform(orders, customers)
load(clean)
```

ETL Pipeline Implementation Example

Example 3 — Incremental ETL Pipeline with Validation

```
import pandas as pd, logging
from sqlalchemy import create_engine

logging.basicConfig(level=logging.INFO)
engine = create_engine("sqlite:///sales.db")

def extract(last_id):
    query = f"SELECT * FROM orders WHERE id > {last_id}" # incremental extract
    return pd.read_sql(query, engine)

def transform(df):
    df = df.dropna(subset=["price", "qty"]) # ensure critical fields exist
    df["revenue"] = df["price"] * df["qty"]
    return df[df["revenue"] > 0] # basic data validation

def load(df):
    df.to_sql("sales_fact", engine, if_exists="append", index=False)

def run_pipeline():
    data = extract(last_id=1000)
    clean = transform(data)
    load(clean)
    logging.info("ETL pipeline executed")

run_pipeline()
```

Automating workflows with schedulers

- ✓ In data engineering many tasks must run **repeatedly at specific intervals**.
 - Examples include refreshing datasets, running ETL pipelines, generating reports, or retraining models.
 - Executing these tasks manually is inefficient and error-prone.
- ✓ **Workflow automation** refers to the process of scheduling programs or scripts so that they run **automatically at predefined times or intervals**.
 - This ensures that data pipelines operate continuously without manual intervention.
- ✓ Common automated tasks in data pipelines include:
 - Running **ETL pipelines periodically**
 - Updating **analytical datasets**
 - Generating **reports and dashboards**
 - Performing **data backups**
 - Running **model retraining jobs**
 - Schedulers are tools that **trigger scripts or programs based on time or events**.

Two common scheduling approaches are:

- **CRON (system-level scheduler)**
- **Python-based scheduler libraries such as schedule**

CRON

- ✓ **CRON** is a **Unix/Linux system scheduler** used to run commands or scripts automatically at specified times.
 - It is widely used in production environments to automate backend tasks and data pipelines.
- ✓ It is most commonly used to automate system maintenance or administration tasks, but it can be used for any purpose that requires regular, scheduled command execution.
- ✓ A CRON job consists of:
 - A **time schedule expression**
 - A **command or script to execute**
- ✓ CRON runs in the background and executes tasks according to the defined schedule.

Automating workflows with schedulers (Cont..)

Components of CRON

✓ Cron Daemon (crond)

- The **cron daemon** is the background service that continuously runs in the system.
- It checks scheduled jobs every minute.

*"/etc/crontab" is a system-wide configuration file for **System-Wide Crontab***

✓ Crontab (Cron Table)

- **Crontab** is the configuration file where scheduled jobs are stored.
- Each user can have their **own crontab file**, which lists commands and their execution times.
- Example crontab entry: `0 * * * * python3 pipeline.py`

Command	Purpose
<code>crontab -e</code>	Create or edit the current user's cron jobs
<code>crontab -l</code>	List all cron jobs
<code>crontab -r</code>	Remove all cron jobs
<code>crontab -u username -e</code>	Edit another user's cron jobs (admin only)

Setting Up and Managing Cron Jobs

✓ Creating a User Cron Job

- To create or edit a cron job for the current user, use: `crontab -e`
- This command opens the **user's crontab file** in the system's default editor (e.g., **Vi**, **Nano**).

✓ To schedule a job, add a new line using the standard cron syntax.

- Example: Run a backup script every day at **3:00 AM**.
- `0 3 * * * /home/user/backup.sh`
- After adding the job, save and exit the editor.
- The cron service automatically detects changes, so restarting cron is not required.
- `crontab -e` also performs basic syntax checking when saving.

✓ Creating a System-Wide Cron Job

- User cron jobs affect only the current user.
- For **system-level scheduling**, the file `/etc/crontab` is used.

Automating workflows with schedulers (Cont..)

CRON (Cont..)

✓ CRON Time Expression Format

- CRON schedules are defined using **five-time fields** followed by the command to execute.

*	*	*	*	*
min	hour	day of month	month	day of week
(0-59)	(0-23)	(1-31)	(1-12)	(0-6) (0=Sunday)
-	range of values		MON-FRI,1-5	
*	any value in range		*	
/	step values		*/10	
,	value list separator		0,20,40	

✓ Basic Scheduling Examples

- `* * * * * python3 pipeline.py` → Runs every minute
- `0 * * * * python3 pipeline.py` → Runs at the **start of every hour**.
- `0 2 * * * python3 pipeline.py` → Run daily at 2 AM.
- `0 3 * * 0 python3 weekly_report.py` → Runs every Sunday at **3 AM**.

Automating workflows with schedulers (Cont..)

CRON (Cont..)

✓ Using Step Intervals:

- Step values allow tasks to run at **regular intervals**.

- `*/5 * * * * python3 pipeline.py`

➔ Runs every **5 minutes**.

- `0 */2 * * * python3 pipeline.py`

➔ Run every 2 hours

- Runs at: 00:00, 02:00, 04:00, ...

✓ Using Lists and Ranges

- CRON allows **multiple values and ranges**.

- `0 6,12,18 * * * python3 pipeline.py`

➔ Runs at: 6 AM 12 PM 6 PM

- `0 9-17 * * * python3 update_dashboard.py`

➔ Runs every hour

from **9 AM to 5 PM (Working hours)**

- `0 8 * * 1-5 python3 send_report.py`

➔ Runs at **8 AM Monday–**

Friday.

Automating workflows with schedulers (Cont..)

✓ Special Characters in CRON Syntax

- CRON syntax supports special characters to define flexible scheduling patterns.

Symbol	Meaning	Example
*	Represents every value of the field	* in hour → every hour
,	Specifies multiple values (list)	1,3,5 → Monday, Wednesday, Friday
-	Specifies a range of values	9-17 → from 9 AM to 5 PM
/	Specifies interval increments	*/10 → every 10 minutes

✓ CRON Shortcut Strings

- CRON also provides shortcut strings that replace the five time fields.

Shortcut	Meaning	Equivalent CRON
@reboot	Run once when system starts	—
@yearly / @annually	Run once every year at Jan 1, 12:00 AM	0 0 1 1 *
@monthly	Run on the 1st day of every month at 12:00 AM	0 0 1 * *
@weekly	Run every Sunday at 12:00 AM	0 0 * * 0
@daily / @midnight	Run every day at 12:00 AM	0 0 * * *
@hourly	Run at the beginning of every hour	0 * * * *

CRON (Cont..)

✓ Viewing and Managing CRON Jobs

- `crontab -l` → View scheduled jobs
- `crontab -e` → Edit CRON jobs
- `crontab -r` → Remove all jobs

✓ Logging Output from CRON Jobs

- CRON jobs can redirect output to log files for monitoring.
- `0 2 * * * python3 etl_pipeline.py >> pipeline.log 2>&1`
 - `>> pipeline.log` → append output to log file
 - `2>&1` → redirect errors to the same file

✓ Best Practices When Using CRON

- Use **absolute paths** for scripts and Python interpreter
- Log outputs for **debugging and monitoring**

Automating workflows with schedulers (Cont..)

Schedule

- ✓ The **schedule** library is a lightweight Python package used to create simple schedulers directly inside Python programs.
 - Unlike CRON, which runs at the operating system level, schedule runs **inside a Python application**.
- ✓ It is useful for small automation tasks and lightweight pipelines.
- ✓ Installation: `pip install schedule`
- ✓ Features:
 - Time-based execution
 - Integration with Python functions - Any Python function can be scheduled as a job.
 - Lightweight implementation - No external services or system configuration is required.

Automating workflows with schedulers (Cont..)

Schedule (Cont..)

✓ Major Functions of the Python schedule Library

Function / Method	Purpose	Example
schedule.every()	Defines the base scheduling interval for a job	<code>schedule.every(5).minutes.do(job)</code>
.seconds	Schedule job to run every specified number of seconds	<code>schedule.every(10).seconds.do(job)</code>
.minutes	Run job every specified number of minutes	<code>schedule.every(15).minutes.do(job)</code>
.hours	Run job every specified number of hours	<code>schedule.every(2).hours.do(job)</code>
.days	Run job every specified number of days	<code>schedule.every(2).days.do(job)</code>
.day.at("HH:MM")	Run job daily at a specific time	<code>schedule.every().day.at("02:00").do(job)</code>
.monday(), .tuesday(), etc.	Schedule job on a specific weekday	<code>schedule.every().monday.do(job)</code>
.do(function)	Assign the function to execute when the job runs	<code>schedule.every().hour.do(run_etl)</code>
schedule.run_pending()	Execute all scheduled jobs whose time has arrived	<code>schedule.run_pending()</code>
schedule.cancel_job(job)	Cancel a previously scheduled job	<code>schedule.cancel_job(job)</code>
schedule.clear()	Remove all scheduled jobs	<code>schedule.clear()</code>
schedule.get_jobs()	Return a list of all scheduled jobs	<code>schedule.get_jobs()</code>

Automating workflows with schedulers (Cont..)

Schedule (Cont..)

- ✓ A typical scheduling workflow includes three steps:
 - Define a **function representing the task**
 - Register the task using **schedule rules**
 - Run a loop that continuously **checks and executes scheduled jobs**

```
import schedule
import time

def job():
    print("Running scheduled task")

# schedule the job every minute
schedule.every(1).minutes.do(job)

while True:
    schedule.run_pending() # run scheduled tasks
    time.sleep(1) # wait before checking again
```

Automating workflows with schedulers (Cont..)

Schedule (Cont..)

✓ Scheduling Jobs at Specific Times

- Jobs can also be scheduled at **specific times during the day**.
- Example:** run every day at 2:00 AM.

```
import schedule, time

def run_pipeline():
    print("Running ETL pipeline")

schedule.every().day.at("02:00").do(run_pipeline)

while True:
    schedule.run_pending()
    time.sleep(60)
```

✓ Scheduling Jobs at Different Intervals

- `schedule.every(10).minutes.do(run_pipeline)`
- `schedule.every().hour.do(run_pipeline)`
- `schedule.every().day.at("09:00").do(run_pipeline)`
- `schedule.every().friday.at("18:00").do(run_pipeline)`

Automating workflows with schedulers (Cont..)

Schedule (Cont..)

✓ Scheduling Multiple Tasks

- Multiple workflows can be scheduled within the same program.

```
import schedule, time

def etl_job():
    print("Running ETL pipeline")

def report_job():
    print("Generating report")

schedule.every().hour.do(etl_job)
schedule.every().day.at("18:00").do(report_job)

while True:
    schedule.run_pending()
    time.sleep(30)
```

✓ Using schedule with ETL Pipelines

```
import schedule, time

def run_etl():
    print("Executing ETL pipeline")
    # call extract(), transform(), load() functions

schedule.every().day.at("01:00").do(run_etl)

while True:
    schedule.run_pending()
    time.sleep(60)
```

✓ Managing Scheduled Jobs: Cancelling and removing all

- `job = schedule.every(10).minutes.do(run_pipeline)`
- `schedule.cancel_job(job)`
- `schedule.clear()`

Logging, monitoring, and error handling in pipelines

- ✓ The ETL pipeline can be failed due to:
 - **Data sources fail, missing files, invalid data, database connection failures, or transformation errors.**
- ✓ The difference between a good ETL pipeline and a great one:
 - how gracefully your system handles the unexpected errors.
- ✓ Reliable data pipelines require mechanisms to record execution events, detect failures, and manage errors.
 - **Logging** records pipeline activities and debugging information.
 - **Monitoring** tracks pipeline execution and system performance.
 - **Error handling** prevents failures from crashing the entire workflow.
- ✓ **Logging, monitoring, and error handling** help developers:
 - track pipeline execution
 - detect failures quickly
 - debug problems efficiently
 - maintain reliability of automated workflows

Logging in Data Pipelines

- ✓ **Logging** is the process of recording events that occur during the execution of a program.
- ✓ In data pipelines, logs provide detailed information about:
 - each stage of the pipeline, including data extraction, transformation steps, loading status, and errors,
- ✓ Logs are useful for:
 - debugging failures
 - tracking pipeline progress
 - auditing pipeline operations
 - diagnosing performance issues

Logging, monitoring, and error handling in pipelines

Logging in Data Pipelines in Python

- ✓ Python provides a built-in **logging module** to record messages from programs. It allows logs to be printed to the console or saved in files.

- ✓ Logging Levels

- The logging module provides several levels to classify messages.

Level	Purpose
DEBUG	Detailed diagnostic information
INFO	General pipeline progress messages
WARNING	Potential issues that do not stop execution
ERROR	Failures that affect a task
CRITICAL	Severe failures that may terminate the pipeline

```
import logging                                # Import logging module

logging.basicConfig(                           # Configure logging settings
    level=logging.INFO,                       # Log messages of INFO level and above
    format="%(asctime)s - %(levelname)s - %(message)s" # Include time, level, message
)

logging.info("ETL pipeline started")          # Informational message
logging.warning("Some values are missing")    # Warning about potential issue
logging.error("Database connection failed")   # Error message indicating failure
```

Logging, monitoring, and error handling in pipelines

Example - Logging in an ETL Pipeline

```
import logging
import pandas as pd

logging.basicConfig(level=logging.INFO)

def extract():
    logging.info("Extracting data from CSV")
    return pd.read_csv("sales.csv")

def transform(df):
    logging.info("Transforming data")
    df["revenue"] = df["price"] * df["quantity"]
    return df

def load(df):
    logging.info("Loading data to output file")
    df.to_csv("processed_sales.csv", index=False)

data = extract()
data = transform(data)
load(data)

logging.info("ETL pipeline completed")
```

Monitoring Data Pipelines

- ✓ **Monitoring** refers to continuously observing pipeline execution to detect issues such as failures, delays, or abnormal behavior.
- ✓ Monitoring helps ensure that pipelines:
 - run at scheduled times
 - complete successfully
 - process expected data volumes
- ✓ Monitoring techniques include:
 - checking log files
 - tracking execution time
 - verifying data quality
 - generating alerts for failures
- ✓ Key aspects monitored in ETL systems include:
 - pipeline start and completion time
 - execution duration
 - number of processed records
 - success or failure of each stage
 - system resource usage

Monitoring Examples

✓ Monitoring Execution Time

- Monitoring runtime helps detect **performance bottlenecks**.

```
import time
import logging

logging.basicConfig(level=logging.INFO)

start = time.time()

def run_pipeline():
    logging.info("Running pipeline...")
    time.sleep(2) # simulate work

run_pipeline()

end = time.time()

logging.info(f"Execution time: {end-start:.2f} seconds")
```

✓ Monitoring Data Quality

- Pipelines often validate data before loading.

```
import logging
import pandas as pd

df = pd.read_csv("sales.csv")

if df.isnull().sum().sum() > 0:
    logging.warning("Dataset contains missing values")

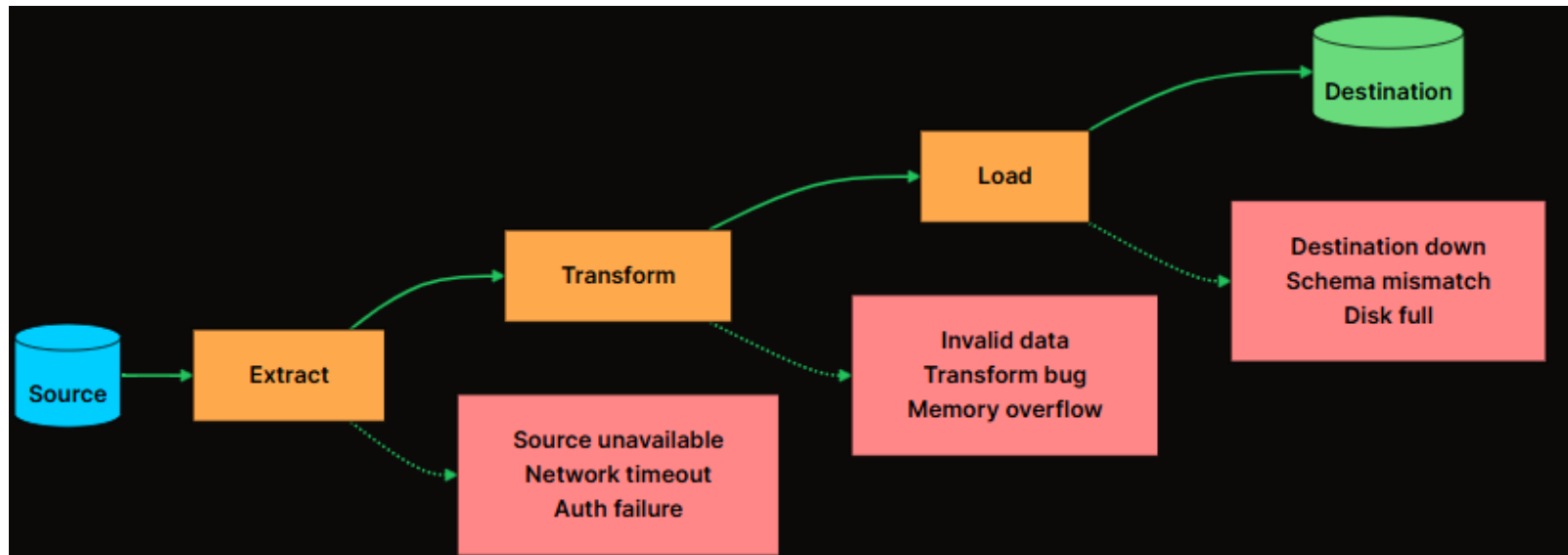
if len(df) == 0:
    logging.error("Dataset is empty")
```

Error Handling in Pipelines

- ✓ **Error handling** is the process of detecting and managing runtime errors to prevent the entire pipeline from crashing.
- ✓ Python uses **try-except blocks** to handle exceptions.
- ✓ Benefits:
 - prevents pipeline failure
 - allows graceful recovery
 - records error details in logs
- ✓ Typical errors in ETL pipelines include:
 - file not found errors
 - database connection failures
 - invalid data formats
 - transformation errors
- ✓ Python pipelines use **try-except blocks** to catch exceptions and handle them gracefully.

Logging, monitoring, and error handling in pipelines

Example – Error types in ETL



Logging, monitoring, and error handling in pipelines

ETL Pipeline – Error Handling Example

✓ Basic Error Handling Example

```
import logging

logging.basicConfig(level=logging.INFO)

try:
    file = open("data.csv")
    logging.info("File opened successfully")
except FileNotFoundError:
    logging.error("Input file not found")
```

Error Handling in ETL Pipeline →

```
logging.basicConfig(level=logging.INFO)

def extract():
    try:
        logging.info("Reading input data")
        return pd.read_csv("sales.csv")
    except Exception as e:
        logging.error(f"Extraction failed: {e}")
        return None

def transform(df):
    try:
        df["revenue"] = df["price"] * df["quantity"]
        return df
    except Exception as e:
        logging.error(f"Transformation failed: {e}")

def load(df):
    try:
        df.to_csv("output.csv", index=False)
        logging.info("Data loaded successfully")
    except Exception as e:
        logging.error(f>Loading failed: {e}")

data = extract()

if data is not None:
    data = transform(data)
    load(data)
```

Data storage and retrieval strategies for pipeline

- ✓ Data storage and retrieval strategies define how data produced by pipelines is structured, organized, stored, and accessed for downstream analytics, reporting, and machine learning tasks.
- ✓ Key objectives:
 - Persist processed datasets reliably for future use.
 - Organize data logically to support automation and maintainability.
 - Scale storage capacity as data volumes grow.
- ✓ Core design dimensions:
 - **Data modeling** — representing data entities and relationships in storage.
 - **Storage technologies** — databases, data warehouses, or data lakes selected by workload.
 - **File formats** — Parquet, Avro, or CSV balancing efficiency with query performance.
 - **Partitioning & Indexing** — dividing and indexing data to accelerate retrieval and reduce query costs.
 - **Retrieval mechanisms** — how downstream systems query and consume stored data.

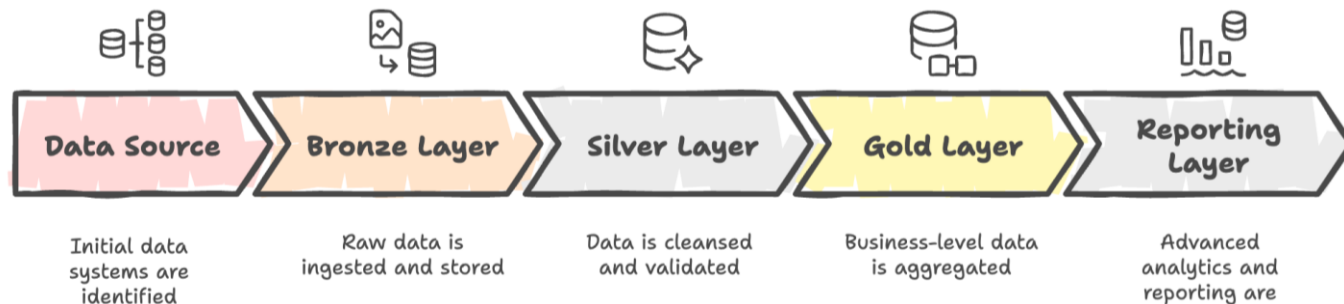
Data storage and retrieval strategies for pipeline

Storage Layers in Pipelines

✓ Typical pipeline workflow:

- Data Sources → Ingestion → Storage Layer → Processing → Analytics / ML / Reporting

Medallion Architecture



Storage Layer	Description	Purpose	Example Technologies
Raw Layer (Bronze)	Stores data exactly as ingested from sources	Enables auditing, debugging, and data reprocessing	Amazon S3, Hadoop HDFS, Google Cloud Storage
Processed Layer (Silver)	Stores cleaned and structured datasets after ETL	Supports analytics and ML pipelines	Apache Spark storage, Delta Lake, Parquet datasets
Analytics Layer (Gold)	Stores aggregated or feature-engineered datasets optimized for analysis	Used by BI dashboards and ML models	Snowflake, BigQuery, Amazon Redshift

Data storage and retrieval strategies for pipeline

Example transformation pipeline

✓ Example:

- Load JSON data from APIs → Clean & Transform (*Remove null, datetime string into datetime object*) → Analysis (*Aggregate total revenue by date*) → **Store (parquet file)**

```
import pandas as pd

# Load raw API data stored in the raw layer
raw = pd.read_json("data/raw/api_sales.json")

# Clean dataset and enforce schema for processed layer
processed = raw.dropna(subset=["product_id"])
processed["date"] = pd.to_datetime(processed["date"])

# Aggregate data for analytics layer (e.g., dashboard-ready dataset)
analytics = processed.groupby("date")["revenue"].sum().reset_index()

# Store analytics dataset in columnar format for efficient queries
analytics.to_parquet("data/analytics/daily_sales.parquet")
```

Data storage and retrieval strategies for pipeline

Choosing Database Type

- ✓ When selecting a database model, we should analyze:
 - Schema flexibility, query patterns, transaction requirements, latency constraints

Database Model	Data Structure	Advantages	Technologies	Typical Use
Relational Database	Structured tables with predefined schema	Strong consistency and SQL querying	PostgreSQL, MySQL	Transaction datasets
Columnar Database	Data stored column-wise	Fast aggregation queries	ClickHouse, BigQuery	Analytical pipelines
Document Database	JSON-like document storage	Flexible schema	MongoDB, CouchDB	API data ingestion
Key-Value Database	Simple key-value pairs	Extremely fast lookup	Redis, DynamoDB	Caching

Data storage and retrieval strategies for pipeline

Deciding Data Storage Formats

- ✓ Important considerations before selecting a format:
 - Dataset size, query patterns, schema complexity, processing framework compatibility

Format	Storage Type	Advantages	Limitations	Technologies
CSV	Row-based text format	Simple and widely supported	Large file size, slow parsing	Pandas, Excel
JSON	Semi-structured hierarchical format	Flexible schema	Inefficient for large datasets	MongoDB
Parquet	Columnar binary format	Fast analytics queries, high compression	Not human readable	Spark, Pandas
ORC (Optimized Row Columnar)	Columnar format for big data systems	Efficient compression and indexing	Big data ecosystem focused	Hive

Data storage and retrieval strategies for pipeline

Efficient Format - Parquet

✓ Why Parquet Dominates Analytics Pipelines

- It is columnar binary format with fast analytics queries, high compression
- **Column pruning:** only reads requested columns → 10–100× faster than CSV
 - Columnar formats allow analytics engines to **scan only required columns** rather than **entire rows**, significantly improving query performance.

```
import pandas as pd

# Load dataset stored in CSV format
df = pd.read_csv("sales.csv")

# Convert to columnar Parquet format for faster analytical queries
df.to_parquet(
    "data/processed/sales.parquet",
    compression="snappy" # compression reduces storage size and improves I/O
)
```

Data storage and retrieval strategies for pipeline

Data Partitioning Strategies

- ✓ Partitioning divides large datasets into **smaller logical segments stored separately**, enabling faster data retrieval and scalable processing.
- ✓ Benefits of partitioning:
 - Reduces query scanning time, enables distributed processing, organizes large datasets logically, improves query performance in data warehouses

Partition Strategy	Description	Example
Time-based partitioning	Data grouped by date or time attributes	year=2026 / month=03
Key-based partitioning	Data grouped by categorical attribute	country=NP
Range partitioning	Data divided by numerical intervals	revenue ranges
Hash partitioning	Data distributed evenly using hash function	user_id hashing

Data storage and retrieval strategies for pipeline

Example – Storage Implementation

```
import sqlite3, pandas as pd, json

data = [{"id": 1, "name": "Alice", "score": 95.5},
        {"id": 2, "name": "Bob", "score": 82.3}]

df = pd.DataFrame(data)

# 1. SQLite – relational database
conn = sqlite3.connect("pipeline.db")
df.to_sql("scores", conn, if_exists="replace", index=False)
pd.read_sql("SELECT * FROM scores", conn)

# 2. CSV – flat file storage
df.to_csv("scores.csv", index=False)
pd.read_csv("scores.csv")

# 3. Parquet – columnar format (efficient for analytics)
df.to_parquet("scores.parquet", index=False)
pd.read_parquet("scores.parquet")

# 4. JSON – document storage
with open("scores.json", "w") as f:
    json.dump(data, f)
```

Data storage and retrieval strategies for pipeline

Indexing Strategies for Data Retrieval

- ✓ Indexes allow databases to **locate records quickly without scanning entire datasets.**
- ✓ Reasons indexing is important in pipelines:
 - reduces query execution time, improves filtering performance, accelerates join operations
- ✓ Engineers choose retrieval strategies based on:
 - Pipeline latency requirements, dataset size, update frequency, analytical workload patterns

Retrieval Strategy	Description	Pipeline Usage
Batch Retrieval	Load complete dataset periodically	Scheduled analytics
Incremental Retrieval	Load only newly generated records	Continuous ETL
Query-based Retrieval	Retrieve filtered subsets using SQL	Dashboards
Streaming Retrieval	Process real-time incoming data	Real-time analytics

Data storage and retrieval strategies for pipeline

Indexing Strategies for Data Retrieval

```
import pandas as pd

df = pd.read_parquet("data/pipeline_dataset")

# Batch retrieval (entire dataset)
batch_data = df

# Filtered retrieval (key partition query)
country_data = df[df.country=="NP"]

# Range-based retrieval
high_sales = df[df.revenue_range=="high"]

# Incremental retrieval (new records after pipeline run)
incremental = df[df.timestamp > "2025-01-10"]

print(len(batch_data), len(country_data), len(high_sales), len(incremental))
```

Data storage and retrieval strategies for pipeline

Example - Pipeline

- ✓ **Problem:** From a transaction file `transactions.csv`, compute and maintain **total high-value sales (>700) per region incrementally**.

```
def ingest(file):
    return pd.read_csv(file, parse_dates=["timestamp"]) # Load raw data

def transform(df):
    # Partitioning strategies for storage:
    df["year"] = df.timestamp.dt.year # time-based
    df["region_key"] = df.region # key-based
    df["amount_range"] = pd.cut(df.amount, [0, 700, 1e9], labels=["low", "high"]) # range-based
    df["hash"] = df.customer_id % 2 # hash-based
    return df

def store(df, path="data/pipeline"):
    df.to_parquet(path, partition_cols=["year", "region_key", "amount_range", "hash"], index=False)

def retrieve(path="data/pipeline"):
    df = pd.read_parquet(path) # Batch retrieval
    high_sales = df[df.amount_range=="high"] # Range-based retrieval
    return high_sales

def analyze(df_subset):
    return df_subset.groupby("region_key")["amount"].sum().reset_index() # Query-based aggregation

# ----- Orchestrator -----
df = ingest("transactions.csv") # 1) Ingest
df = transform(df) # 2) Transform / Partition
store(df) # 3) Store partitioned data
high_sales = retrieve() # 4) Retrieve only high-value sales
result = analyze(high_sales) # 5) Aggregate per region
```

Automated report generation

- ✓ **Definition:** Programmatically producing formatted, human-readable documents from data — without manual intervention
 - A report is the *final delivery artifact* of a pipeline: raw data → insights → formatted document → stakeholder
 - Automated = triggered by schedule, pipeline completion, or event; no human in the loop
- ✓ **Pipeline Flow:**
 - Data Sources → Ingestion → Transformation → Storage → Analytics → Report Generation
- ✓ **Key roles of automated reporting:**
 - Data communication, Automation, Integration

Aspect	Report	Dashboard
Delivery	Push (scheduled, emailed)	Pull (user opens browser)
Format	Static snapshot (Excel/HTML/PDF)	Interactive, live
Audience	Management, regulators, clients	Analysts, operations
Tools	Python, Jinja2, openpyxl	Grafana, Superset, Streamlit

Automated report generation

Python Libraries

✓ XCEL

- **openpyxl** — read/write, full features, charts, styles
- **xlsxwriter** — write-only, faster, better formatting API
- **xlrd** — read legacy .xls only
- **pyxlsb** — read binary .xlsb files

✓ HTML

- **Jinja2** — templating engine, industry standard
- **pandas** — `df.to_html()` for quick tables
- **dominate** — programmatic HTML construction
- **yagmail** — HTML email delivery

✓ PDF

- **WeasyPrint** — HTML+CSS → PDF

- **pdfkit** — wkhtmltopdf wrapper (requires binary)
- **ReportLab** — low-level programmatic layout, powerful
- **fpdf2** — lightweight, pure Python
- **borb** — modern, object-oriented PDF library
- **pypdf** — read/merge/split existing PDFs

✓ CHARTS (embed anywhere)

- **matplotlib** — static charts → PNG → embed
- **plotly** — interactive (HTML) or static (PDF/Excel)
- **seaborn** — statistical charts on matplotlib
- **altair** — declarative, Vega-based

Automated report generation

Python Libraries

Format	Library	Role
Excel	<code>xlsxwriter</code>	Write-only, fast, rich formatting + charts
Excel	<code>openpyxl</code>	Read/write, modify existing files
HTML	<code>Jinja2</code>	Template engine — separates layout from data
HTML	<code>pandas</code>	<code>df.to_html()</code> for quick inline tables
PDF	<code>ReportLab</code>	Programmatic layout, precise control, pure Python
PDF	<code>WeasyPrint</code>	HTML+CSS → PDF conversion
Charts	<code>matplotlib</code>	Static PNG → embed in any format
Charts	<code>plotly</code>	Interactive (HTML) or static export

Automated report generation (Contd..)

Excel

- ✓ Excel supports **tables, charts, formulas, formatting, and multiple sheets**, making it suitable for business reporting.
 - Used for **financial reports, KPI dashboards, operational summaries**.
 - Supports **structured tables, formulas, and charts**.
 - Easily shared with non-technical stakeholders.
 - Can be generated automatically after ETL or analytics stages.
- ✓ Python Libraries for Excel Report Generation

Library	Primary Use	Advantages	Limitations
pandas	Quick export	Very simple	Limited formatting
openpyxl	Full Excel control	Modify sheets, styles, formulas	Slightly verbose
xlsxwriter	Advanced formatting	Charts, formatting	Cannot read files

Automated report generation (Contd..)

Excel

✓ Generating Excel with pandas

- This is the **simplest method** when the goal is exporting a dataset.
- **Characteristics**
 - Minimal code. Direct export from DataFrame, Suitable for simple reports

```
# Step 1 – analytics result from pipeline
data = {
    "Product": ["Laptop", "Phone", "Tablet"],
    "Sales": [1200, 900, 700],
    "Region": ["East", "West", "East"]
}

df = pd.DataFrame(data)

# Step 2 – aggregate analytics result
summary = df.groupby("Product")["Sales"].sum().reset_index()

# Step 3 – export directly to Excel
# pandas automatically creates the workbook
summary.to_excel("sales_report.xlsx", index=False)
```

Automated report generation (Contd..)

Excel (Contd..)

✓ Generating Excel with pandas

- This is the **simplest method** when the goal is exporting a dataset.
- **Characteristics**
 - Minimal code. Direct export from DataFrame, Suitable for simple reports
- **Limitations**
 - Limited formatting
 - Difficult to add formulas
 - Limited control over workbook structure

```
# Step 1 - analytics result from pipeline
data = {
    "Product": ["Laptop", "Phone", "Tablet"],
    "Sales": [1200, 900, 700],
    "Region": ["East", "West", "East"]
}

df = pd.DataFrame(data)

# Step 2 - aggregate analytics result
summary = df.groupby("Product")["Sales"].sum().reset_index()

# Step 3 - export directly to Excel
# pandas automatically creates the workbook
summary.to_excel("sales_report.xlsx", index=False)
```

Automated report generation (Contd..)

Excel - XlsxWriter

✓ XlsxWriter Based Excel

Reporting

- XlsxWriter is designed for **creating formatted Excel reports with charts and styling.**
- Unlike openpyxl, it **cannot read existing Excel files**, but it is very powerful for **creating formatted reports from scratch.**
- **Capabilities**
 - Chart generation
 - Conditional formatting
 - Cell formatting
 - Table creation
 - Dashboard style reports

```
# analytics result from pipeline
df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr"],
    "Sales": [200, 340, 280, 400]
})

# create Excel writer using XlsxWriter engine
writer = pd.ExcelWriter("sales_dashboard.xlsx", engine="xlsxwriter")

# export dataframe
df.to_excel(writer, sheet_name="Sales", index=False)

# access workbook objects
workbook = writer.book
worksheet = writer.sheets["Sales"]

# create chart object
chart = workbook.add_chart({"type": "column"})

# configure chart series
chart.add_series({
    "name": "Monthly Sales",
    "values": "=Sales!B2:B5"
})

# insert chart into worksheet
worksheet.insert_chart("E2", chart)

# save report
writer.close()
```

Automated report generation (Contd..)

Excel - OpenPyXL

- ✓ OpenPyXL is the **most flexible library for programmatic Excel manipulation.**
- ✓ It allows pipelines to:
 - create Excel files
 - modify existing reports
 - insert formulas
 - apply formatting
 - manage multiple sheets
 - process large Excel reports
 - This makes it ideal for **automated reporting pipelines.**
- ✓ **Architecture of OpenPyXL**
 - OpenPyXL follows a hierarchical object model.

Object	Description
Workbook	Entire Excel file
Worksheet	Individual sheet
Cell	Single data element

Automated report generation (Contd..)

Excel - OpenPyXL

✓ Core OpenPyXL Functions

- **Workbook Level Functions**

Function	Description
Workbook()	Create new Excel file
load_workbook()	Load existing file
save()	Save workbook
create_sheet()	Create new worksheet
remove()	Delete sheet

- **Styling and Formatting Functions**

Feature	Function
Font style	Font()
Alignment	Alignment()
Background color	PatternFill()
Borders	Border()

- **Worksheet Level Methods**

Method	Purpose
append()	Add row
cell()	Access specific cell
iter_rows()	Iterate rows
iter_cols()	Iterate columns
max_row	Total rows
max_column	Total columns
Method	Purpose

- **Formula Support**

Formula	Purpose
SUM()	Total values
AVERAGE()	Mean
COUNT()	Count records

Automated report generation (Contd..)

Excel – Example Pipeline

- **Example reporting pipeline** using OpenPyXL and pandas.

```
import pandas as pd
from openpyxl import Workbook
from openpyxl.styles import Font

# -----
# Step 1 – Load processed data from pipeline output
# -----
df = pd.read_csv("transactions.csv")

# -----
# Step 2 – Perform analytics aggregation
# Example: revenue by product
# -----
summary = df.groupby("product")["amount"].sum().reset_index()

# -----
# Step 3 – Create Excel workbook
# -----
wb = Workbook()

# Raw data sheet
raw_sheet = wb.active
raw_sheet.title = "Transactions"

# Summary analytics sheet
summary_sheet = wb.create_sheet("Summary")
```

```
# Step 4 – Write raw data into Excel
# -----
raw_sheet.append(df.columns.tolist()) # header

for row in df.values:
    raw_sheet.append(list(row))

# -----
# Step 5 – Write aggregated analytics results
# -----
summary_sheet.append(summary.columns.tolist())

for row in summary.values:
    summary_sheet.append(list(row))

# -----
# Step 6 – Add formatting to header rows
# -----
for cell in raw_sheet[1]:
    cell.font = Font(bold=True)

for cell in summary_sheet[1]:
    cell.font = Font(bold=True)

# -----
# Step 7 – Add Excel formula for total revenue
# -----
summary_sheet["D2"] = "=SUM(B2:B10)"

# -----
# Step 8 – Save final automated report
# -----
wb.save("automated_sales_report.xlsx")
```

Automated report generation (Contd..)

HTML

- ✓ Automated HTML reporting refers to programmatically generating structured web-based reports from pipeline outputs.
 - HTML documents can be viewed in browsers, embedded in dashboards, or shared via email.
 - HTML reports are widely used because they support interactive visualization, styling, and web integration.

Feature	Explanation
Web compatible	Can be opened in any browser
Rich formatting	CSS styling support
Visualization	Integrates with JavaScript charts
Easy sharing	Can be embedded in emails
Lightweight dashboards	Suitable for automated monitoring

- ✓ Pipelines typically combine:
 - **pandas** → convert analytical results to tables
 - **Jinja2** → dynamic HTML templating engine
 - **Plotly / Matplotlib** → chart generation
- ✓ The most important component is **Jinja2 templating**, which enables **dynamic HTML generation from pipeline data**.

Automated report generation (Contd..)

HTML

✓ Python Approaches for HTML Report Generation

Approach	Library	Use Case	Complexity
Direct DataFrame export	pandas	Simple tables	Low
HTML templating	Jinja2	Structured reports	Medium
Visualization dashboards	Plotly / Bokeh	Interactive reports	High

✓ Generating HTML Tables using Pandas

- The simplest method is converting a DataFrame directly into an HTML table.
- DataFrame.to_html()

Parameter	Purpose
index	Include index column
classes	Add CSS classes
border	Table border size
justify	Alignment

```
# convert dataframe to HTML table
html_table = df.to_html(index=False)

# write HTML report
with open("sales_report.html","w") as f:
    f.write("<h1>Sales Report</h1>")
    f.write(html_table)
```

Automated report generation (Contd..)

HTML – Jinja2 Templating Engine

- ✓ Jinja2 is a powerful Python templating engine used to generate HTML dynamically from structured data.
 - It separates presentation layer (HTML) from data processing logic (Python).
- ✓ Jinja2 templates use **special syntax embedded inside HTML**.

Syntax	Purpose
<code>{{ variable }}</code>	Output variable
<code>{% ... %}</code>	Logic statement
<code>{# ... #}</code>	Comment

- ✓ Jinja2 Template Control Structures

```
{% if revenue > 1000 %}  
<p>High revenue product</p>  
{% endif %}
```

- **Variable Assignment**

```
{% set total = 0 %}
```

- **Looping**

```
{% for row in data %}  
<tr>  
<td>{{ row.product }}</td>  
<td>{{ row.amount }}</td>  
</tr>  
{% endfor %}
```

Automated report generation (Contd..)

Jinja2 Functions and Methods

• Environment Methods

Method	Purpose
<code>Environment()</code>	Create template environment
<code>get_template()</code>	Load template file
<code>from_string()</code>	Compile template from string

```
template = env.get_template("report.html")
```

✓ Filters

```
{{ variable | filter }}
```

- Filters modify variables inside templates.

Filter	Purpose
<code>upper</code>	Convert to uppercase
<code>lower</code>	Convert to lowercase
<code>round</code>	Round numeric values
<code>safe</code>	Render HTML safely

• Template Methods

Method	Purpose
<code>render()</code>	Render template with data
<code>generate()</code>	Stream output
<code>module</code>	Access template functions

```
html = template.render(data=records)
```

✓ Macros

- Macros allow **reusable template functions**.

```
{% macro table_row(product,revenue) %}  
<tr>  
<td>{{ product }}</td>  
<td>{{ revenue }}</td>  
</tr>  
{% endmacro %}
```

```
{{ table_row("Laptop",2500) }}
```

Automated report generation (Contd..)

HTML – Jinja2 Templating Engine

✓ Jinja2 Environment and Template Loading

```
from jinja2 import Environment, FileSystemLoader
import pandas as pd

# pipeline analytics output
df = pd.DataFrame({
    "product": ["Laptop", "Phone", "Tablet"],
    "revenue": [2500, 900, 700]
})

# convert dataframe to dictionary records
records = df.to_dict(orient="records")

# initialize template environment
env = Environment(loader=FileSystemLoader("templates"))

# load HTML template
template = env.get_template("report.html")

# render HTML with dynamic values
html = template.render(data=records)

# save report
with open("sales_report.html", "w") as f:
    f.write(html)
```

```
<html>

<head>
<title>Sales Analytics</title>
</head>

<body>

<h1>Sales Summary</h1>

<table border="1">

<tr>
<th>Product</th>
<th>Revenue</th>
</tr>

{% for row in data %}
<tr>
<td>{{ row.product }}</td>
<td>{{ row.revenue }}</td>
</tr>
{% endfor %}

</table>

</body>

</html>
```

Automated report generation (Contd..)

Embedding Charts into HTML Reports

- ✓ Charts can be embedded using Plotly.

Function	Purpose
<code>px.bar()</code>	Create bar chart
<code>fig.to_html()</code>	Convert chart to HTML
<code>write_html()</code>	Save standalone HTML

```
import pandas as pd
import plotly.express as px
from jinja2 import Template

# analytics result
df = pd.DataFrame({
    "product": ["Laptop", "Phone", "Tablet"],
    "revenue": [2500, 900, 700]
})

# create chart
fig = px.bar(df, x="product", y="revenue")

chart_html = fig.to_html(full_html=False)

# convert table
table_html = df.to_html(index=False)
```

```
# template
template = Template("""
<html>

<body>

<h1>Sales Report</h1>

<h2>Revenue Chart</h2>
{{ chart | safe }}

<h2>Revenue Table</h2>
{{ table | safe }}

</body>

</html>
""")

html = template.render(chart=chart_html, table=table_html)

with open("sales_report.html", "w") as f:
    f.write(html)
```

Automated report generation (Contd..)

PDF

- ✓ Automated PDF reporting is the process of **programmatically generating structured, printable documents from pipeline outputs.**
- ✓ Two dominant approaches:
 - **Direct PDF generation (ReportLab)**
 - **HTML-to-PDF conversion (WeasyPrint / pdfkit)**

Library	Approach	Key Features
ReportLab	Programmatic PDF creation	Precise layout control
WeasyPrint	HTML → PDF conversion	Uses HTML/CSS
pdfkit	HTML → PDF using wkhtmltopdf	Web-style rendering
FPDF / PyFPDF	Lightweight PDF generator	Simple text-based reports

Automated report generation (Contd..)

PDF Generation using FPDF

✓ FPDF is a lightweight library suitable for **simple automated reports**.

✓ **Core Capabilities**

- text writing, table-like layouts, page management

✓ **Limitations**

- manual layout design, difficult complex formatting
- limited styling

Method	Purpose
add_page()	Create new page
set_font()	Set font style
cell()	Write text cell
multi_cell()	Multi-line text
output()	Save PDF

```
from fpdf import FPDF
import pandas as pd

# analytics output from pipeline
df = pd.DataFrame({
    "Product": ["Laptop", "Phone", "Tablet"],
    "Revenue": [2500, 900, 700]
})

pdf = FPDF()

pdf.add_page()
pdf.set_font("Arial", size=12)

# report title
pdf.cell(200, 10, "Sales Report", ln=True)
```

```
# table header
for col in df.columns:
    pdf.cell(40, 10, col, 1)
pdf.ln()

# table rows
for row in df.values:
    for item in row:
        pdf.cell(40, 10, str(item), 1)
    pdf.ln()

pdf.output("sales_report.pdf")
```

Automated report generation (Contd..)

PDF Generation using ReportLab

- ✓ ReportLab is the **most powerful Python library for programmatic PDF generation.**

```
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
from reportlab.lib.styles import getSampleStyleSheet

styles = getSampleStyleSheet()

doc = SimpleDocTemplate("report.pdf")

elements = []

elements.append(Paragraph("Sales Report", styles["Title"]))
elements.append(Spacer(1,20))
elements.append(Paragraph("Automated pipeline generated report.", styles["Normal"]))

doc.build(elements)
```

Automated report generation (Contd..)

PDF Generation using ReportLab

✓ Document Creation

Class	Purpose
SimpleDocTemplate	Create PDF document
BaseDocTemplate	Advanced layout control

```
doc = SimpleDocTemplate("report.pdf")
```

✓ Text and Styling

Class	Purpose
Paragraph	Styled text block
ParagraphStyle	Text formatting
getSampleStyleSheet()	Default styles

```
Paragraph("Sales Report", styles["Title"])
```

Component	Purpose
SimpleDocTemplate	Main document container
Paragraph	Styled text
Table	Tabular data
Spacer	Layout spacing
Image	Insert images

✓ Layout Components

Component	Purpose
Spacer	Vertical spacing
PageBreak	New page
Image	Insert figures

Automated report generation (Contd..)

PDF Generation using ReportLab

- ✓ Creating Tables with ReportLab

```
from reportlab.platypus import SimpleDocTemplate, Table
import pandas as pd

df = pd.DataFrame({
    "Product": ["Laptop", "Phone", "Tablet"],
    "Revenue": [2500, 900, 700]
})

data = [df.columns.tolist()] + df.values.tolist()

doc = SimpleDocTemplate("table_report.pdf")

table = Table(data)

doc.build([table])
```

Automated report generation (Contd..)

PDF Generation using ReportLab

- ✓ Inserting Charts and Images

Python

```
import matplotlib.pyplot as plt

plt.bar(df["Product"], df["Revenue"])
plt.savefig("chart.png")
```

Insert into PDF:

Python

```
from reportlab.platypus import Image

elements.append(Image("chart.png"))
```

Automated report generation (Contd..)

PDF Generation using ReportLab

- ✓ An example of an automated pipeline step generating a PDF report.

```
import pandas as pd
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table
from reportlab.lib.styles import getSampleStyleSheet

# Load processed pipeline data
df = pd.read_csv("transactions.csv")

# analytics computation
summary = df.groupby("product")["amount"].sum().reset_index()

styles = getSampleStyleSheet()

# convert dataframe to table format
table_data = [summary.columns.tolist()] + summary.values.tolist()

elements = []

# report title
elements.append(Paragraph("Automated Sales Report", styles["Title"]))
elements.append(Spacer(1,20))

# table
elements.append(Table(table_data))

# create pdf
doc = SimpleDocTemplate("sales_report.pdf")
doc.build(elements)
```

Case study: End-to-end automated analytics pipeline

✓ Student Task!